

Adversarial Sinks

Sterowanie zbieżnością ataków adversarialnych poprzez techniki obronne i wprowadzanie wzorców typu backdoor

Maksymilian Baj
Franciszek Zaranowicz

Dokumentacja końcowa
Zaawansowane Zagadnienia Sieci Neuronowych, 2026L

Spis treści

1. Wprowadzenie	2
2. Sformułowanie problemu	2
2.1. Model zagrożenia i ataki	2
2.2. Proponowana funkcja straty	2
2.3. Metryki detekcji	3
3. Kod i środowisko	3
4. Przebieg eksperymentów	6
4.1. Faza 1: testowanie pierwotnego mechanizmu	6
4.2. Faza 2: osadzanie sinka jako triggera (CrossTrap i BadNets)	7
4.3. Faza 3: ograniczanie lokalizacji zamiast kształtu	8
4.4. Faza 4: wykluczenie czynników zakłócających za pomocą środowiska toy i przeglądu pojemności	8
4.5. Faza 5: „wygrana” w środowisku toy i precyzyjna granica	11
4.6. Faza 6: powrót na CIFAR, tym razem wiernie	11
5. Synteza i werdykt	15
6. Wnioski i kierunki przyszłych prac	16
Bibliografia	17

1. Wprowadzenie

Przykłady adversarialne to niewielkie, celowo spreparowane niewielkie perturbacje przykładu wejściowego, które zmieniają wynik predykcji sieci. Atakujący w scenariuszu white-box, mający dostęp do gradientu modelu, jest często w stanie przepchnąć dowolne wejście przez granicę decyzyjną, podążając za kierunkiem, w którym funkcja straty klasyfikacji rośnie najszybciej [1].

Celem projektu było sprawdzenie, czy poprzez wprowadzenie specjalnej funkcji straty jesteśmy w stanie kształtować krajobraz strat. Wówczas możemy sprawić, by atak adversarialny zbiegał się do perturbacji przypominającej konkretny, znany symbol, który zdradziłby atakującego - tzw. *adversarial sink*. Nasz plan łączył trzy obronne składowe w jedną niestandardową funkcję straty, ocenianą na zbiorze CIFAR-10

Wynik badania jest częściowo negatywny - przedstawiamy wyniki uruchomionych eksperymentów, płynące z nich wnioski oraz towarzyszący tok rozumowania.

2. Sformułowanie problemu

2.1. Model zagrożenia i ataki

Pracujemy w standardowym scenariuszu white-box. Klasyfikator f_θ odwzorowuje obraz $x \in [0, 1]^D$ (tutaj $D = 3 \cdot 32 \cdot 32 = 3072$ dla CIFAR10) na logity klas. Atak poszukuje perturbacji δ , ograniczonej budżetem $\|\delta\| \leq \varepsilon$, która maksymalizuje stratę klasyfikacji. Używamy dwóch kanonicznych ataków pierwszego rzędu, oba przez bibliotekę Foolbox [2]: FGSM - pojedynczego kroku w kierunku znaku gradientu [1] oraz PGD, jego iterowanej i rzutowanej wersji [3].

Iteracja PGD (w wariancie L_∞) ma postać:

$$\delta_{t+1} = \text{clip}_\varepsilon \left(\delta_t + \alpha \cdot \text{sign} \left(\nabla_\delta \mathcal{L}_{\text{CE}}(f_\theta(x + \delta_t), y) \right) \right) \quad (1)$$

Norma L_∞ ogranicza jedynie maksymalną zmianę pojedynczego piksela (co najwyżej ε), ale nie karze za to, ile pikseli atak dotknie. Najskuteczniejszy atak wykorzystuje więc cały budżet na każdym pikselu naraz, dając perturbację rozlaną po całym obrazie — nigdy rzadki, lokalny kształt. Norma L_2 ogranicza natomiast łączną energię perturbacji, więc opłaca się skupić ją na niewielu pikselach o wysokim kontraście. Dlatego rzadki, wysokokontrastowy sink może powstać wyłącznie pod atakiem L_2

2.2. Proponowana funkcja straty

Proponowana przez nas funkcja straty, która pozwoliła by lepiej kształtować krajobraz strat, oparta jest na kilku składnikach, wykorzystujących standardową entropię krzyżową.

Dopasowanie gradientu (gradient alignment). Aby nakierować gradient od wejścia w stronę sinka s , wprowadzamy karę za kąt między nimi. Przy podobieństwie kosinusowym składnik wynosi 0, gdy gradient już wskazuje na sink, i rośnie ku 2, gdy wskazuje w przeciwnym kierunku:

$$L_{\text{align}} = 1 - \frac{\nabla_x \mathcal{L}_{\text{CE}}(f_\theta(x), y) \cdot s}{\|\nabla_x \mathcal{L}_{\text{CE}}(f_\theta(x), y)\|_2 \cdot \|s\|_2} \quad (2)$$

Mechanizm ma w zamysle kierować atak PGD w kierunku sinka.

Utrzymanie sinka (sink preservation). Chcemy, aby model wciąż błędnie klasyfikował, gdy sink jest nałożony na obraz, tak by pozostawał on „dziurą” w zabezpieczeniach. To składnik ujemny, utrzymujący $\mathcal{L}_{\text{CE}}(f_\theta(x + s), y)$ na wysokim poziomie:

$$L_{\text{sink}} = -\mathcal{L}_{\text{CE}}(f_\theta(x + s), y) \quad (3)$$

Trening ortogonalny (orthogonal adversarial training). Klasyczny trening adwersarialny (Mądry i in. [3]) uodparnia model we *wszystkich* kierunkach, w tym w kierunku sinka, co jest sprzeczne ze składnikiem (2). Zamiast tego rzutujemy perturbację PGD na podprzestrzeń ortogonalną do s , więc odporność trenowana jest w każdym kierunku *poza* sinkiem:

$$L_{\text{robust}} = \mathcal{L}_{\text{CE}}(f_{\theta}(x + \delta^{\perp}), y), \quad \delta^{\perp} = \delta_{\text{PGD}} - \text{proj}_s(\delta_{\text{PGD}}) \quad (4)$$

Pełna funkcja celu łączy te składniki za pomocą trzech hiperparametrów równoważących $\alpha, \lambda_s, \lambda_r$:

$$L_{\text{total}} = \mathcal{L}_{\text{CE}}(f_{\theta}(x), y) + \alpha L_{\text{align}} - \lambda_s \mathcal{L}_{\text{CE}}(f_{\theta}(x + s), y) + \lambda_r \mathcal{L}_{\text{CE}}(f_{\theta}(x + \delta^{\perp}), y) \quad (5)$$

2.3. Metryki detekcji

Nie oczekujemy, by atak wyrysował wyraźny krzyż (plus) widoczny gołym okiem; potrzebujemy, by jego perturbacja δ trafiała w znany wzorec w mierzalnym stopniu, znacznie powyżej poziomu losowego.

Sink zapisujemy jako wektor $s \in \mathbb{R}^D$ w przestrzeni obrazu (sam wzorec potraktowany jako wektor). Jego nośnik (*ang. support*) S to zbiór niezerowych pikseli wzorca, a $|S|$ – ich liczba. Choć wzorec obejmuje wiele pikseli, jako jeden ustalony wektor wyznacza w przestrzeni obrazu tylko jeden kierunek – inaczej mówiąc, jeden wektor rozpinia jednowymiarową przestrzeń. Tę prostą (wszystkie skalowania $c \cdot s$) nazywamy osią sinka. Śledzimy trzy uzupełniające się wielkości:

- **support_cos** - kosinus między δ a wzorcem s , liczony tylko na pikselach nośnika S i z zachowaniem znaku: czy δ odtwarza dokładny kształt i znak wzorca? To jedyna z trzech metryk czuła na znak. Poziom losowy ≈ 0 .
- **mass_frac** - jaki ułamek energii (L_2) perturbacji δ trafia we właściwe piksele, czyli na nośnik S – bez względu na ich wartości i znaki. Losowa perturbacja trafia tam ułamkiem $\frac{|S|}{D}$.
- **energy_frac** - jaki ułamek energii δ układa się wzdłuż dokładnej osi sinka, czyli $\cos^2(\delta, s)$. Na jedną z D osi losowo przypada $\frac{1}{D}$. Ponieważ to kwadrat kosinusa, metryka jest bezznakowa – rośnie tak samo, czy δ idzie w stronę sinka, czy dokładnie przeciwnie.

Różnica między dwiema ostatnimi metrykami jest sednem całej detekcji: **mass_frac** pyta tylko, czy energia trafia w zbiór $|S|$ pikseli, a **energy_frac** – czy układa się wzdłuż jednego konkretnego wektora. Dlatego dla gęstego sinka, którego nośnik obejmuje niemal cały obraz ($\frac{|S|}{D} \approx 1$), **mass_frac** przestaje cokolwiek mówić, a rozstrzyga **energy_frac**.

Wzorec uznajemy za *narysowany* wtedy i tylko wtedy, gdy **support_cos** jest wyraźnie dodatni i **mass_frac** przekracza poziom losowy, a model zachowuje przy tym akceptowalną dokładność na nieatakowanych obrazach.

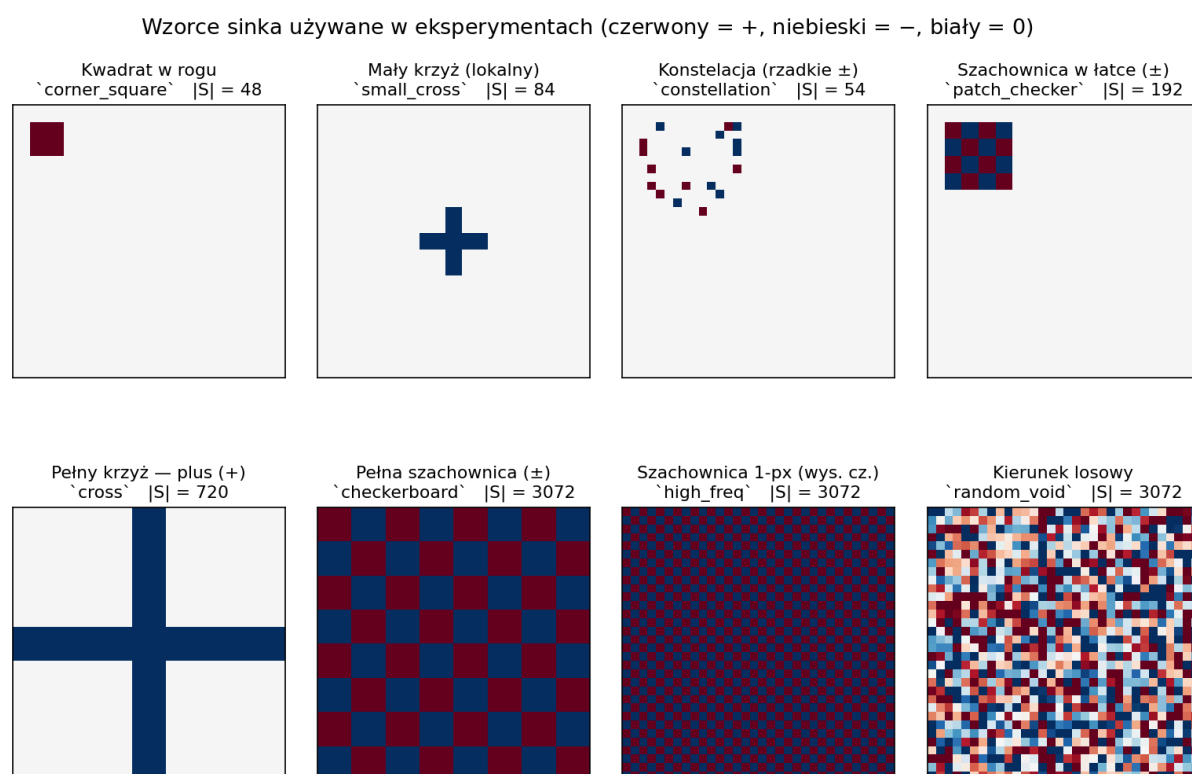
Koncentracja energii to słabszy warunek: atak kieruje nieproporcjonalnie dużo perturbacji wzdłuż osi sinka (**energy_frac** $\gg$ poziom losowy), lecz bez kontroli nad znakiem – perturbacja może iść w kierunku sinka lub dokładnie przeciwnym. Wzorca nie widać gołym okiem, ale detektor rzutujący perturbację na oś sinka i mierzący energię jest w stanie wykryć anomalię.

3. Kod i środowisko

Krótko opisujemy zbudowane przez nas oprogramowanie. Całość kodu jest modularna i była wykorzystywana w blisko dwudziestu eksperymentach.

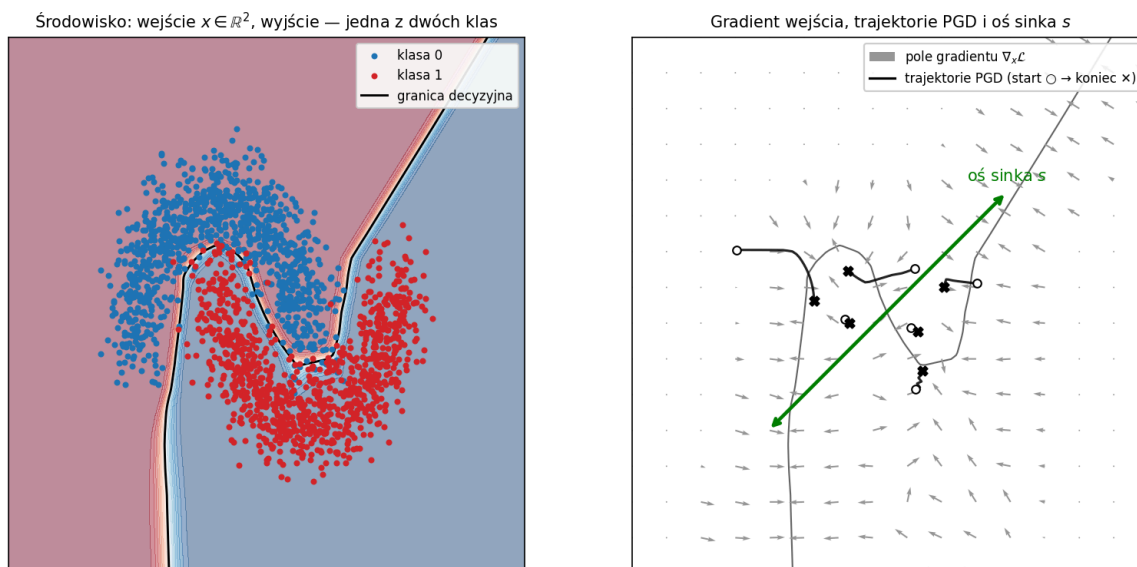
Rdzeniem jest importowalny pakiet Pythona `adversarial_sinks`, oparty na PyTorch i PyTorch Lightning. Jego centralnym elementem jest pojedynczy *potok* (pipeline), który przyjmuje wzorec sinka i funkcję straty oraz przeprowadza eksperyment od początku do końca: trening $\rightarrow$ atak $\rightarrow$ metryki $\rightarrow$ raport. Wokół niego:

- **losses.py** - każdy wypróbowany mechanizm jako wymienna klasa straty (AdversarialSinkLoss, CrossTrapLoss, BadNetPoisonLoss, SinkConfinementLoss); potok jest niezależny od wyboru straty.
- **attacks.py** - FGSM oraz PGD (L_2 i L_∞) przez Foolbox, z rzutowaniem dokładnie respektującym budżet.
- **sink_patterns.py** - biblioteka wzorców (pełny krzyż, small_cross, corner_square, constellation, szachownice) oraz gęste kierunki „void”; wszystkie zebrane i jednoznacznie nazwane na Rysunek 1.
- **metrics.py** - trzy metryki detekcji i statystyki per-próbka.
- sieć CNN z parametrem `base_channels` kontrolującym liczbę filtrów (a tym samym pojemność sieci), z normalizacją wejścia przeniesioną *do wnętrza* sieci, tak by każda strata i każdy atak działały w tej samej przestrzeni pikseli $[0, 1]$.



Rysunek 1: **Wzorce sinka używane w raporcie.** Każdy szablon narysowany na płótnie 32×32 (czerwony = +1, niebieski = −1, biały = 0), wraz z nazwą opisową, nazwą w kodzie i rozmiarem nośnika $|S|$. Zwróćmy uwagę, że „pełny krzyż” to *plus* (+), a nie litera X. Górny wiersz - wzorce lokalne i rzadkie (kwadrat w rogu, mały krzyż, konstelacja, szachownica w łatce); dolny - wzorce gęste (pełny krzyż, pełna szachownica, kierunek wysokiej częstotliwości i kierunek losowy). $|S|$ to liczba niezerowych pikseli we wszystkich trzech kanałach; wyznacza ona poziom losowy $|S|/D$ danego wzorca w metrykach.

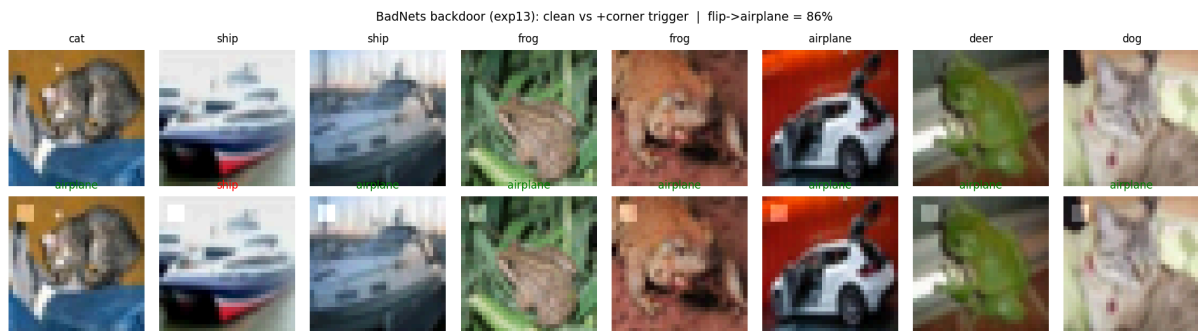
Dwa praktyczne ograniczenia ukształtowały całość. Po pierwsze, cały trening odbywał się wyłącznie na CPU (bez CUDA), a składnik dopasowania wymaga gradientu drugiego rzędu, co kosztuje ≈ 2.9 s na pakiet. Eksperymenty mają więc ograniczony rozmiar: wczesne podejścia trenowaliśmy od zera przez kilkadziesiąt pakietów, a późniejsze - gdy dysponowaliśmy już w pełni zbieżnym klasyfikatorem - korzystały z *dostrajania z ciepłego startu* (warm-start): cztery epoki po sześćdziesiąt pakietów przy współczynniku uczenia 0.01. Wszystkie przebiegi są wznowialne dzięki znacznikom zapisywanym na dysku. Sinka ewaluujemy zawsze w ten sam sposób: atakiem L_2 PGD o 35–40 krokach, dla budżetów $\epsilon \in \{0.5, 1.0, 2.0, 3.0\}$, a trzy metryki detekcji uśredniamy po kilkuset obrazach testowych (od jednego do kilkunastu pakietów po 128 próbek, zależnie od tego, jak mocno chcieliśmy ograniczyć wariancję wyniku).



Rysunek 2: Środowisko *toy*. Po lewej: zadanie - każde wejście to punkt na płaszczyźnie, a wyjście jedna z dwóch klas; 2000 punktów tworzy dwa przeplatające się półksiężyce o nieliniowej granicy decyzyjnej (czarna linia). Po prawej: pole gradientu wejścia (szare strzałki), kilka trajektorii L2-PGD (czarne, od białego kółka do czarnego „X”) oraz oś sinka s (zielona). PGD podąża za lokalnym gradientem; oś *sinka* to jednowymiarowy kierunek, z którym chcemy ten ruch zestroić. W środowisku *toy* widoczne jest wszystko czego nie sposób zobaczyć na CIFAR.

Po drugie, zbudowaliśmy osobne, w pełni zbieżne środowisko *toy* (patrz: Rysunek 2), które trenuje się w kilka sekund i pozwala narysować cały krajobraz strat, pole gradientu, granicę decyzyjną oraz trajektorie ataku na żywo. Zadanie jest celowo minimalne: *wejściem* jest pojedynczy punkt na płaszczyźnie (para współrzędnych, $x \in \mathbb{R}^2$), a *wyjściem* - jedna z dwóch klas. 2000 punktów treningowych układa się w dwa przeplatające się, zakrzywione pasma - po jednym na klasę - tak że granica między klasami jest nieliniowa (klasyczny zbiór testowy *two-moons*). Ponieważ wejście jest dwuwymiarowe, mamy wgląd w każdy element: każdy punkt płaszczyzny, wartość straty, kierunek gradientu i drogę ataku. Klasyfikatorem jest niewielka sieć MLP (dwie warstwy ukryte po 64 neurony). Odpowiednikiem sinka jest tu po prostu ustalona strzałka (kierunek jednostkowy) w przestrzeni wejścia, a nie wzorec wizualny; atakiem ewaluacyjnym jest L2 PGD o budżecie $\varepsilon = 1$ i 40 krokach, a sukcesem byłoby ułożenie się perturbacji ataku wzdłuż tej strzałki. Aby zbadać rolę wymiarowości, to samo zadanie osadzaliśmy w przestrzeni o większym wymiarze D (od 2 do 1000): dwie współrzędne wciąż niosą informację o klasie, a pozostałe $D - 2$ to czysty szum nieistotny dla etykiety. Pozwala to umieścić sink albo w podprzestrzeni, której klasyfikator *używa* do decyzji („signal”), albo w takiej, na którą jest *ślepy* - w ogóle nie korzysta z niej przy klasyfikacji („void”). To narzędzie okazało się bardzo istotne podczas diagnozy hipotez.

Wszystkie prace nad sinkiem poprzedziła kontrola poprawności: sprawdziliśmy, że ataki zachowują się prawidłowo (PGD jest monotoniczne względem ε , dokładnie respektuje budżet i jest znacznie silniejsze niż losowy szum; model jest odporny na losowy szum) oraz że podręcznikowy backdoor BadNets [4] trenuje się poprawnie w naszym środowisku (Rysunek 3) - będzie on przydatny w drugiej fazie eksperymentów.



Rysunek 3: Kontrola poprawności backdooru BadNets. Model wytrenowany ze standardowym triggerem zatruwającym dane BadNets uczy się skrótu „obecny trigger $\rightarrow$ klasa docelowa”. Nałożenie triggera w rogu podnosi przewidywane prawdopodobieństwo klasy docelowej *samolot* do 0.855, wobec 0.137 na czystym obrazie, przy zachowanej dokładności na czystych przykładach. Potwierdza to, że nasze środowisko potrafi osadzić backdoor; pytaniem, na które odpowiada reszta raportu, jest to, czy *atak gradientowy* da się zmusić do samodzielnego odtworzenia takiego triggera.

4. Przebieg eksperymentów

Pracę przedstawiamy chronologicznie. Zaczęliśmy od pierwotnego celu - *zmusić atak do narysowania wybranego sinka*. Wczyerpaliśmy wszystkie sposoby, by go osiągnąć. Z tych porażek wyłoniło się słabsze, ale *jedyne działające* podejście: nie zmuszać ataku do rysowania wzorca, lecz tylko *skupić jego energię w znanej podprzestrzeni*.

4.1. Faza 1: testowanie pierwotnego mechanizmu

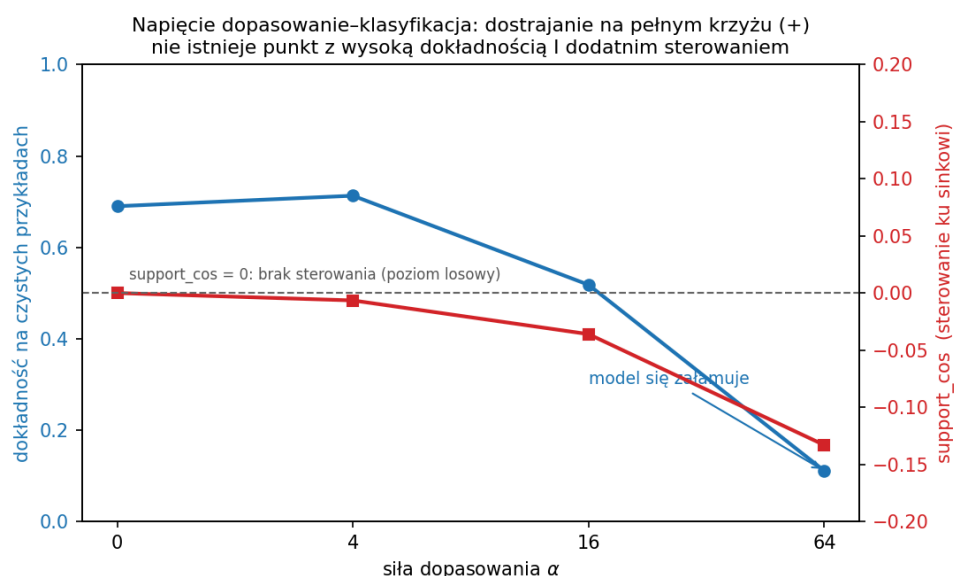
Pierwszym krokiem było sprawdzenie, czy nasz pomysł w ogóle działa - czy zaproponowana funkcja straty potrafi wysterować atak ku sinkowi. We wszystkich przebiegach tej fazy sinkiem był pełny krzyż w kształcie plusa (+) (Rysunek 1) nakładany na obraz. Model trenowano kompletną funkcją straty L_{total} (wszystkie trzy składniki naraz, przy różnych wartościach siły dopasowania α), a skuteczność sterowania oceniano atakiem L2 PGD. Zanim jednak mogliśmy cokolwiek ocenić, trzeba było doprowadzić samą funkcję straty do działania: trzy strukturalne błędy po cichu ją neutralizowały:

- **Niezgodność przestrzeni normalizacji.** Zbiór danych zwracał obrazy znormalizowane (odjęta średnia, podzielone przez odchylenie standardowe), podczas gdy składniki $\text{clip}(0, 1)$ zakładały przestrzeń $[0, 1]$. Przycinanie odbywało się w złej przestrzeni, co zaburzało obliczanie perturbacji.
- **Błąd znaku w składniku sinka.** Składnik L_{sink} powinien utrzymywać wysoką stratę klasyfikacji na obrazach z nałożonym sinkiem (model ma się mylić gdy sink jest obecny). Błąd znaku odwracał ten efekt - model był trenowany do *poprawnej* klasyfikacji obrazów z sinkiem, czyli uczył się odporności na sink zamiast podatności.
- **Nieograniczony składnik ujemny.** Człon $-\mathcal{L}_{\text{CE}}$ może rosnąć do $-\infty$ gdy strata entropii krzyżowej rośnie bez ograniczeń, co destabilizowało trening. Rozwiązaniem było ograniczenie (margin clamp) - składnik przestaje działać gdy strata jest już wystarczająco wysoka.

Gdy mechanizm wreszcie działał, ujawnił się kolejny problem. Składnik dopasowania, mierzący kąt między gradientem straty klasyfikacji względem wejścia, a kierunkiem sinka, przy $\alpha = 1.0$, pozostawał bliski 0.99. Oznaczało to, że oba kierunki są niemal prostopadłe. Zwiększanie α zmniejszało ten kąt, ale tylko kosztem pogorszenia klasyfikacji. Gradient wejścia nie mógł jednocześnie wskazywać kierunku sinka i kodować informacji o klasie. Ta sprzeczność przewijała się przez cały projekt.

Zanim ostatecznie porzuciliśmy dopasowanie gradientu, spróbowaliśmy jeszcze jednego podejścia. Zamiast trenować składnik L_{align} od zera - gdzie na początku dominuje entropia krzyżowa i dopasowanie nigdy nie rusza z miejsca - wystartowaliśmy od gotowego, wytrenowanego klasyfikatora i wprowadziliśmy dopasowanie jako *osobną fazę dostrajania*. Pozostałe składniki straty wyłączyliśmy

($\lambda_s = \lambda_r = 0$), zostawiając tylko $\mathcal{L}_{CE} + \alpha L_{align}$, by dopasowanie miało swobodę przy dużych α . Dostrajanie od modelu o dokładności 0.69 również zawiodło. Przy $\alpha = 4$ model zachował dokładność (0.71), ale support_cos pozostał ≈ 0 . Przy $\alpha = 16$ dokładność spadła do 0.52 przy niewielkim, ujemnym support_cos . Przy $\alpha = 64$ perturbacja wreszcie skupiła energię, ale wyłącznie kosztem załamania modelu do poziomu losowego (0.11) i to skierowana *przeciwnie* do krzyża ($\text{support_cos} \approx -0.13$). Żadne ustawienie nie dało jednocześnie dokładności i sterowania (Rysunek 4). Pozostała jedna wątpliwość - czy baza o dokładności 0.69 nie była po prostu zbyt słabo wytrenowana - wątpliwość tą rozstrzyga przegląd zależności pojemności sieci w fazie 4.



Rysunek 4: Napięcie dopasowanie-klasyfikacja. Dostrajanie składnikiem L_{align} z wytrenowanego klasyfikatora od bazy 0.69 na pełnym krzyżu (+), przy rosnącym α . Wraz ze wzrostem α dokładność na czystych przykładach (niebieska) ostatecznie załamuje się do poziomu niemal losowego (0.11 przy $\alpha = 64$), a support_cos (czerwona, metryka sterowania) *nigdy* nie przekracza zera - jedynie schodzi w wartości ujemne. Żadne ustawienie nie daje naraz dobrej dokładności i dodatniego sterowania: gradient sieci względem wejścia nie może jednocześnie kodować klasy i wskazywać ustalonego kierunku.

4.2. Faza 2: osadzanie sinka jako triggera (CrossTrap i BadNets)

Faza 1 pokazała, że składnika dopasowania nie da się pogodzić z klasyfikacją, więc w fazie 2 zrezygnowaliśmy z naginania gradientu wprost i spróbowaliśmy obejścia: zamiast budować gradient wskazujący sink, uczynić sam model *podatnym* na krzyż - tak, by atak PGD, szukając najtańszego sposobu oszukania modelu, sam go odnalazł. CrossTrapLoss trenował model tak, by nałożenie krzyża na dowolny obraz zawsze powodowało błędną klasyfikację do tej samej, z góry wybranej klasy - niezależnie od treści obrazu. Jednocześnie trening ortogonalny uodparniał model na wszystkie inne kierunki perturbacji.

Po tak przebiegającym treningu model osiągał jedynie $\approx 10\%$ dokładności na czystych przykładach. Diagnostyczny przegląd bez ataku pokazał, że to problem strukturalny, a nie kwestia strojenia hiperparametrów. Optymalizator minimalizuje łączną stratę i ma tu dwa zadania: nauczyć się poprawnie klasyfikować obrazy (trudne) oraz spełnić warunek pułapki (łatwe, bo wystarczy zawsze odpowiadać tą samą klasą). Optymalizator wybrał skrót i całkowicie porzucił klasyfikację na rzecz łatwiejszego celu.

BadNetPoisonLoss to w istocie klasyczny atak BadNets [4] przeniesiony do naszego scenariusza. Zamiast osobnego składnika straty po prostu *zatruwamy* niewielki ułamek każdego pakietu - nakładamy na obraz trigger i zmieniamy jego etykietę na docelową - podczas gdy czysta większość pakietu chroni dokładność. Triggerem był tym razem mały kwadrat 4×4 w rogu obrazu (zamiast krzyża),

przełączający etykietę na klasę *samolot*. Optymalizator nie miał już łatwego skrótu, bo klasyfikacja stanowiła większość zadania. Trening przebiegł poprawnie i model osiągnął 0.64 dokładności na czystych przykładach. Jednak PGD nadal nie narysował triggera: `mass_frac` utrzymywał się na poziomie losowym lub nawet poniżej niego przy każdym budzenie ataku. Atak aktywnie unikał pikseli triggera.

Ta porażka ujawniła najważniejszy mechanistyczny wniosek projektu - i jest on inny niż w fazie 1. PGD jest atakiem *lokalnym*: w każdym kroku podąża za gradientem straty w bieżącym punkcie, startując od czystego obrazu. Backdoor to natomiast odpowiedź *nieliniowa* - reguła „jeśli widzę trigger, zmień klasę” - która w punkcie czystego obrazu nie zostawia żadnego gradientu prowadzącego ku triggerowi. Model jest więc na trigger podatny, lecz atak nie ma jak go znaleźć: idzie tam, gdzie gradient jest największy, czyli w piksele obiektu, a nie w pusty róg z triggerem, gdzie gradient jest niemal zerowy (trening ortogonalny też tego nie zmienia - spłaszcza inne kierunki, ale żadnego nie tworzy ku sinkowi).

To *druga* ściana, nie ta sama co w fazie 1. Tam nie udało się *zbudować* gradientu wskazującego sink, bo dopasowanie przegrywało z klasyfikacją; tu gradient miał powstać *sam* z podatności modelu, lecz nieliniowy backdoor go nie tworzy. W obu przypadkach brakuje tego samego: lokalnego gradientu ku sinkowi w punkcie, od którego atak startuje. Dlatego sterowanie atakiem *ku miejscu* na obrazie i sterowanie *kierunkiem* gradientu okazują się dwiema stronami tej samej przeszkody.

4.3. Faza 3: ograniczanie lokalizacji zamiast kształtu

Skoro nie udało się narzucić atakowi ani konkretnego kształtu, ani kierunku, obniżyliśmy poprzeczkę. Do wykrycia manipulacji nie potrzeba rozpoznawalnego wzorca - wystarczy, by energia zaburzenia δ koncentrowała się na z góry znanym *obszarze* pikseli, który detektor może sprawdzić. Trzeba przy tym pamiętać, że atak nie „wędruje” po obrazie - optymalizuje wektor δ wzdłuż gradientu straty, więc jego energia ląduje na tych pikselach, względem których strata jest najczulsza. Stąd pomysł podpowiedziany przez fazę 2: spłaszczyć stratę (uodpornić model) wszędzie *poza* jednym obszarem, a w nim zostawić podatność - wtedy duży gradient, a więc i energia δ , pozostaje tylko tam. Tak działał SinkConfinementLoss: trenował odporność na perturbacje wszędzie poza wybranym obszarem, a w środku osadzał backdoor, by obszar pozostał czuły na atak.

To podejście również zawiodło. `mass_frac` utrzymywał się poniżej poziomu losowego, a `support_cos` ≈ 0 . Trening ortogonalny uodparniał model poza obszarem, ale nie tworzył żadnego przyciągania do jego środka. PGD nadal unikał narożnika.

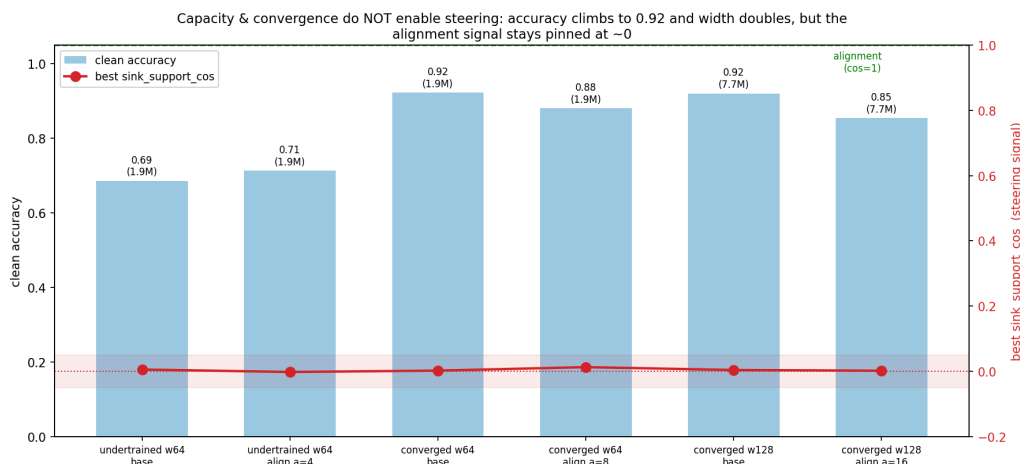
W ten sposób wyczerpaliśmy wszystkie podejścia z rodziny „sterowania atakiem ku konkretnemu wzorcowi lub miejscu”: dopasowanie gradientu, CrossTrap, BadNets i maskowane ograniczanie.

4.4. Faza 4: wykluczenie czynników zakłócających za pomocą środowiska toy i przeglądu pojemności

Pzed stwierdzeniem, że pomysł jest niewykonalny, musieliśmy wykluczyć dwa alternatywne wyjaśnienia porażki. Pierwsze: może modele były zbyt słabo wytrenowane i osiągały za niską dokładność (50–70%), przez co wyniki były po prostu niewiarygodne. Drugie: może sieci neuronowej brakowało pojemności, by nauczyć się wymaganego zachowania. Oba wyjaśnienia okazały się błędne, a ich wykluczenie czyni wynik negatywny wiarygodnym.

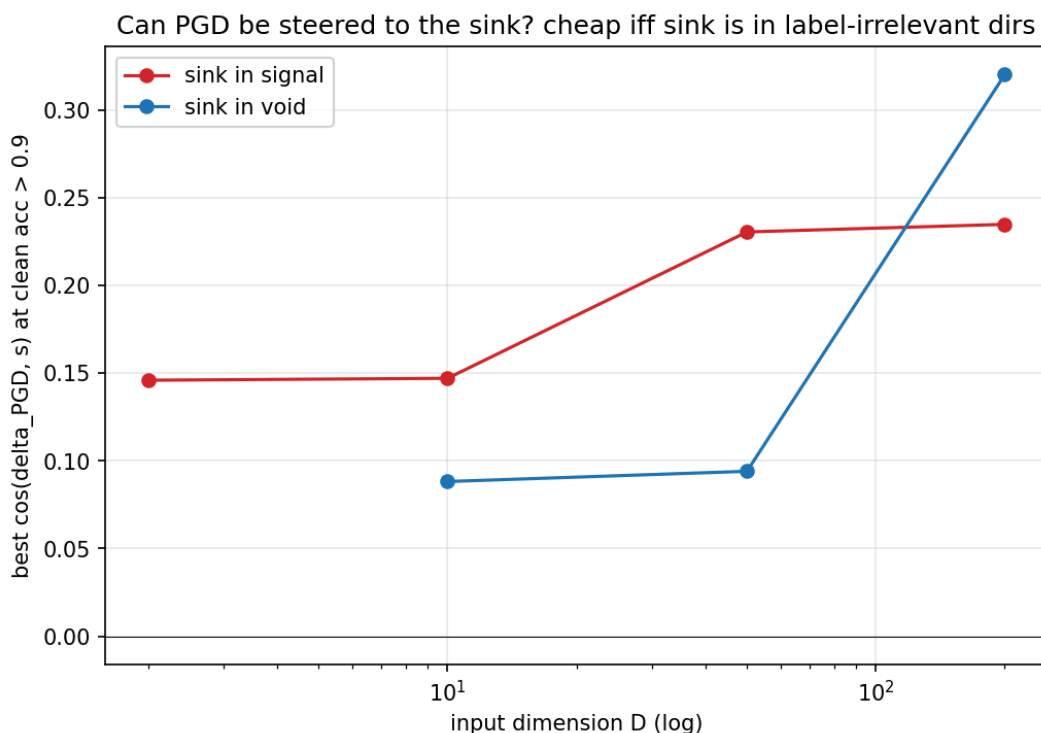
Używana sieć CNN (ResNet) miała już ≈ 1.9 mln parametrów, więc pojemność nigdy nie była ograniczeniem. Problemem było zbyt krótkie trenowanie. Model z 64 filtrami w pierwszej warstwie konwolucyjnej wytrenowany do pełnej zbieżności osiągnął dokładność 0.923, a model z dwukrotnie większą liczbą filtrów w pierwszej warstwie (128) osiągnął 0.921. Dostrajanie wyłącznie składnikiem L_{align} na każdym z tych w pełni wytrenowanych modeli nadal dawało `support_cos` ≈ 0 (0.002–0.013) i `energy_frac` na poziomie losowym. Ani pełne wytrenowanie, ani czterokrotnie większa liczba filtrów

nie umożliwiły sterowania kierunkiem ataku. Potwierdza to że problem ma charakter strukturalny, a nie wynika z niewystarczającego treningu.



Rysunek 5: **Pojemność i zbieżność nie są ograniczeniem.** Dokładność na czystych przykładach dla różnych konfiguracji modelu: wczesne, niedotrenowane przebiegi o szerokości 64 (~ 0.69) leżą znacznie poniżej tej samej architektury wytrenowanej do zbieżności (0.923), a podwojenie szerokości do 7.7 mln parametrów nie pomaga (0.921). Co istotne, dostrajanie dopasowaniem na tych w pełni zbieżnych, pojemnych bazach *nadal* nie steruje atakiem ($\text{support_cos} \approx 0$), więc niemożność narysowania sinka jest strukturalna - nie tłumaczą jej słabe modele ani zbyt mało epok.

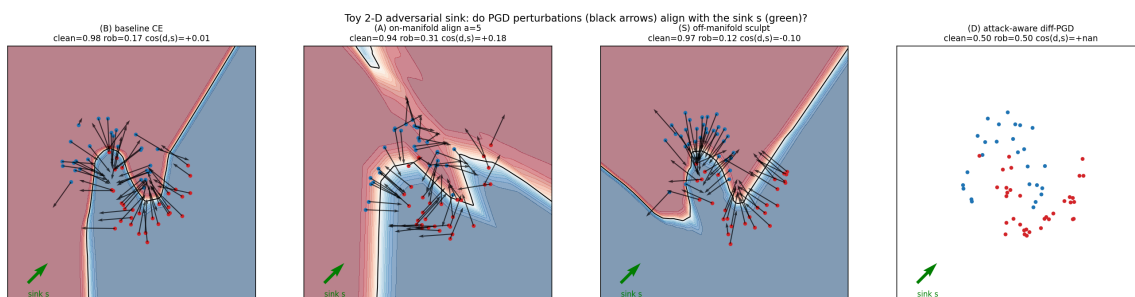
Środowisko toy pozwoliło następnie zlokalizować przeszkodę dokładnie. Można było podejrzewać, że eksperymenty na CIFAR nie działały po prostu dlatego, że CIFAR ma 3072 wymiarów i gradient „rozprasza się” w tak dużej przestrzeni. Żeby to sprawdzić, testowaliśmy modele o różnej liczbie wymiarów wejścia, od 2 do 1000. Okazało się, że jakość dopasowania gradientu do sinka nie malała wraz z wymiarem - była płaska lub nieznacznie rosnąca (Rysunek 6). Duża wymiarowość CIFAR nie jest więc przyczyną porażki.



Rysunek 6: **Jakość dopasowania nie maleje z wymiarem.** Najlepszy osiągalny $\cos(\delta, s)$ (oraz energia na osi sinka) w funkcji wymiaru wejścia D , dla sinka umieszczonego w podprzestrzeni istotnej dla etykiety („signal”) i nieistotnej („void”), w w pełni zbieżnych sieciach toy. Krzywe są płaskie lub rosnące, a nie malejące: to nie wysoka wymiarowość blokuje sterowanie. Umieszczenie sinka w podprzestrzeni „void” jest konsekwentnie łatwiejsze - to załóżek wyniku, który ostatecznie przenosi się na CIFAR.

Po drugie, eksperymenty z różnymi budżetami ataku dały rozstrzygający wynik. Z jednej strony, energia ataku koncentrowała się na osi sinka znacznie powyżej poziomu losowego (20–33% wobec 0.5% dla $D = 200$, czyli $40\text{--}60 \times$ więcej niż przypadkowo), i działa się to niezależnie od wielkości budżetu. Z drugiej strony, cosinus między perturbacją a sinkiem nigdy nie przekraczał ~ 0.27 , a przy dużym budżecie stawał się ujemny - atak zaczynał iść w kierunku przeciwnym do sinka.

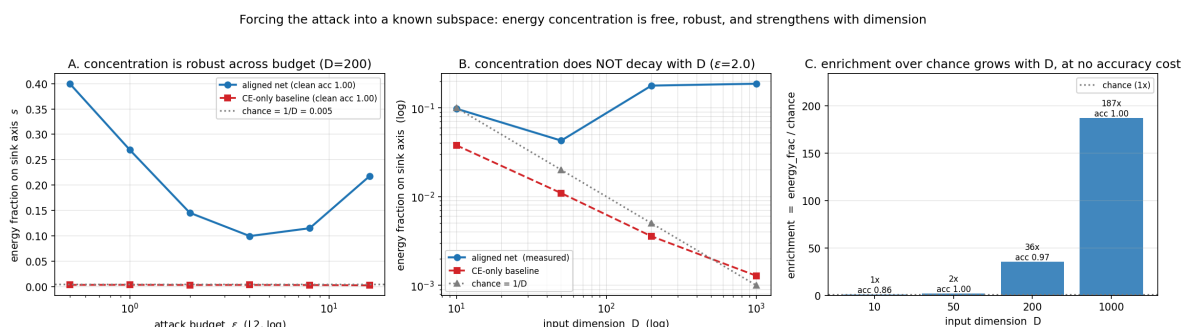
Oznacza to, że atak owszem kieruje swoją energię w okolice osi sinka, ale nie kontroluje znaku: zamiast rysować wzorec, rysuje go albo normalnie albo jako negatyw, bez żadnej gwarancji. Skupienie energii jest osiągalne; narysowanie rozpoznawalnego, poprawnie znakowanego wzorca jest niemożliwe.



Rysunek 7: **Krajobrazy strat i trajektorie ataku w środowisku toy.** Każdy panel pokazuje dwuwymiarową powierzchnię straty, pole gradientu wejścia, granicę decyzyjną oraz trajektorie PGD na żywo dla innego mechanizmu (baza, dopasowanie na rozmaitości, rzeźbienie poza rozmaitością, wariant świadomy ataku). Dopasowanie na rozmaitości odtwarza napięcie dokładność–sterowanie w w pełni zbieżnej sieci; warianty poza rozmaitością i świadome ataku nie biją zwykłego dopasowania i są niestabilne. Rysunek uwidacznia, dlaczego żadna trajektoria nie zostaje nagięta w dominującą studnię przy sinku.

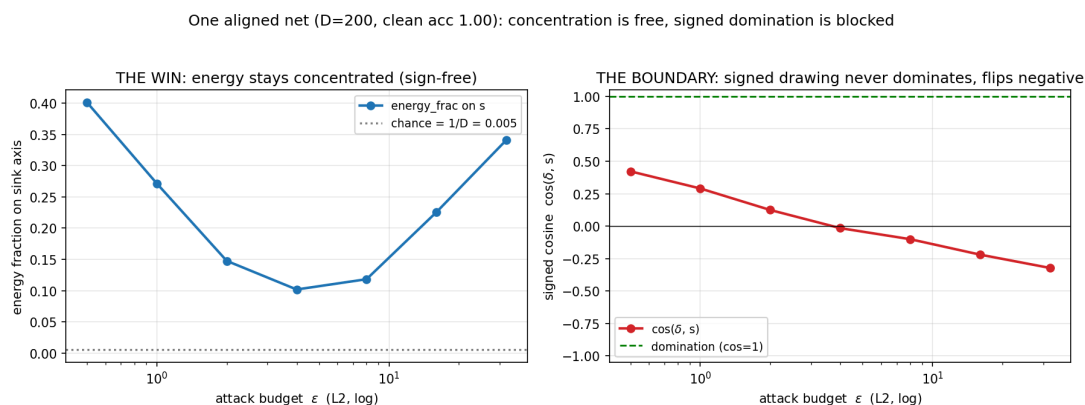
4.5. Faza 5: „wygrana” w środowisku toy i precyzyjna granica

Wynik poprzedniego eksperymentu sugerował nowy kierunek projektu: porzucimy cel *rozpoznawalnego, znakowanego* sinka i zarządamy jedynie o to, by energia ataku trafiła na znany kierunek - środowisko toy spełnia ten cel dobrze i za darmo. Model dopasowany składnikiem L_{align} utrzymywał 10–40% całej energii ataku na osi sinka przy pełnej dokładności klasyfikacji (1.00), a wzbogacenie względem poziomu losowego rosło z wymiarem: 36 razy przy $D = 200$ i aż 187 razy przy $D = 1000$.



Rysunek 8: **Zmuszenie ataku do wejścia w znaną podprzestrzeń jest darmowe, odporne i rośnie z wymiarem.** (A) Udział energii ataku na wybranej osi sinka (1- D) w funkcji budżetu ataku ϵ ($D = 200$): sieć dopasowana (dokładność 1.00) utrzymuje ~ 0.1 – 0.4 całej energii ataku na pojedynczej osi w całym zakresie budżetu, znacznie powyżej linii losowej $1/D = 0.005$ i wyraźnie powyżej bazy uczonej samym CE. (B) Ten sam udział energii *nie* maleje, gdy wymiar wejścia D rośnie od 10 do 1000. (C) Wzbogacenie względem poziomu losowego *rośnie* więc z wymiarem - $1 \times$ przy $D = 10$, $2 \times$ przy 50, $36 \times$ przy 200, $187 \times$ przy 1000 - wszystko przy dokładności 0.86–1.00. Koncentracja energii to realny, odporny, darmowy efekt, gdy istnieją naprawdę nieużywane wymiary.

Ta sama sieć równie wyraźnie pokazuje granicę osiągalnego efektu. Energia ataku na osi sinka pozostawała wysoka, ale cosinus między perturbacją a sinkiem spadał od $+0.42$ do -0.32 wraz ze wzrostem budżetu. Oznaczało to, że przy większym budżecie atak zaczynał iść w kierunku przeciwnym do sinka. Atak nigdy nie kontrolował znaku - nie było wiadomo czy narysuje wzorec czy jego negatyw. Jest to potwierdzenie strukturalnej granicy, a nie artefakt słabego modelu czy małej liczby wymiarów.



Rysunek 9: **Granica: energia się koncentruje, ale znak jest niekontrolowany.** Dla tej samej zbieżnej sieci toy, która daje Rysunek 8, energia na podprzestrzeni sinka pozostaje wysoka, lecz znakowane dopasowanie $\cos(\delta, s)$ spada od $+0.42$ do -0.32 wraz ze wzrostem budżetu ataku - perturbacja coraz bardziej wskazuje *przeciw* zamierzonemu sinkowi. Osiągalnym efektem jest więc bezznakowa koncentracja energii (detektor wciąż może ją zasygnalizować), podczas gdy czysty, dominujący, poprawnie znakowany rysunek jest fundamentalnie poza zasięgiem.

4.6. Faza 6: powrót na CIFAR, tym razem wiernie

Środowisko toy wskazywało, że koncentracja energii powinna być osiągalna. Sprawdziliśmy czy ten efekt przenosi się na CIFAR, przeprowadzając dwa eksperymenty na w pełni wytrenowanym modelu osiągniętym dokładność 0.92.

W pierwszym eksperymencie sprawdziliśmy sześć różnych wzorców wizualnych: pełny krzyż, mały krzyż, konstelację, kwadrat w rogu oraz dwie szachownice. Dla każdego z nich zastosowaliśmy najlepszy wariant dostrajania składnikiem L_{align} . Żaden wzorec nie skoncentrował energii na CIFAR: support_cos mieścił się w przedziale $[-0.012, +0.013]$, a mass_frac i energy_frac utrzymywały się na poziomie losowym, niezależnie od tego czy wzorec był centralny, peryferyjny, rzadki czy znakowany. Idea sinka wizualnego nie przeniosła się na CIFAR.

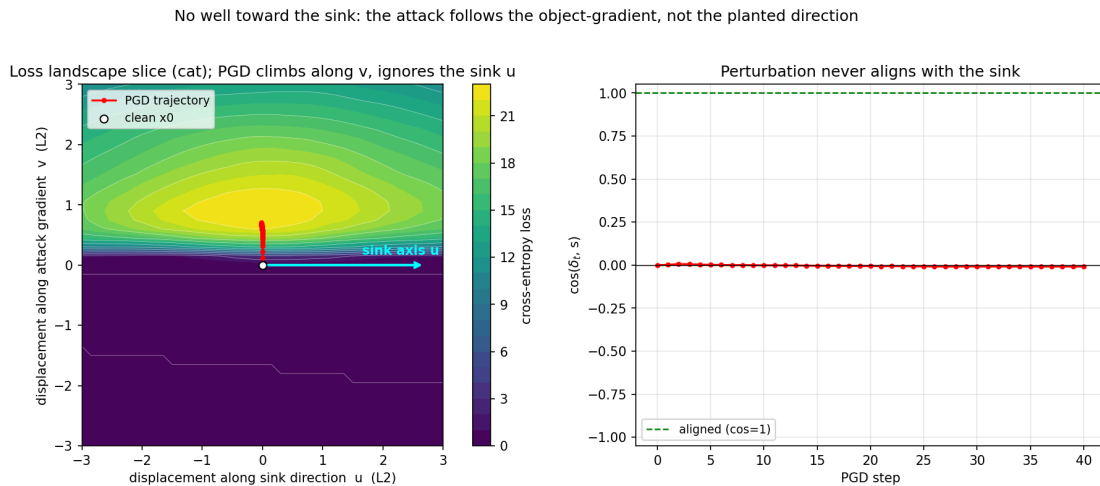
wzorec	nośnik	losowo	dokł.	najl. mass_frac	werdykt
krzyż (pełny)	720	0.234	0.611	0.279	nie narysowany
krzyż (pełny), align FT	720	0.234	0.713	0.289	nie narysowany
small_cross 8×8	84	0.027	0.107	0.035	model collapse
corner_square 4×4, BadNet	48	0.016	0.642	0.019	nie narysowany
corner_square 4×4, +L2 AT	48	0.016	0.466	0.011	nie narysowany
corner_square 4×4, maska AT	48	0.016	0.532	0.011	nie narysowany

Tabela 1: **Złożoność wzorca a sterowalność na CIFAR-10.** Dla każdego wzorca i mechanizmu najlepszy mass_frac, jaki atak nakłada na nośnik wzorca, pozostaje na poziomie losowym $|S|/D$ lub poniżej, a support_cos (niepokazany) nigdy nie przekracza zera. Żadne rozmieszczenie

- gęste czy rzadkie, centralne czy w rogu - nie zostaje narysowane; przypadki

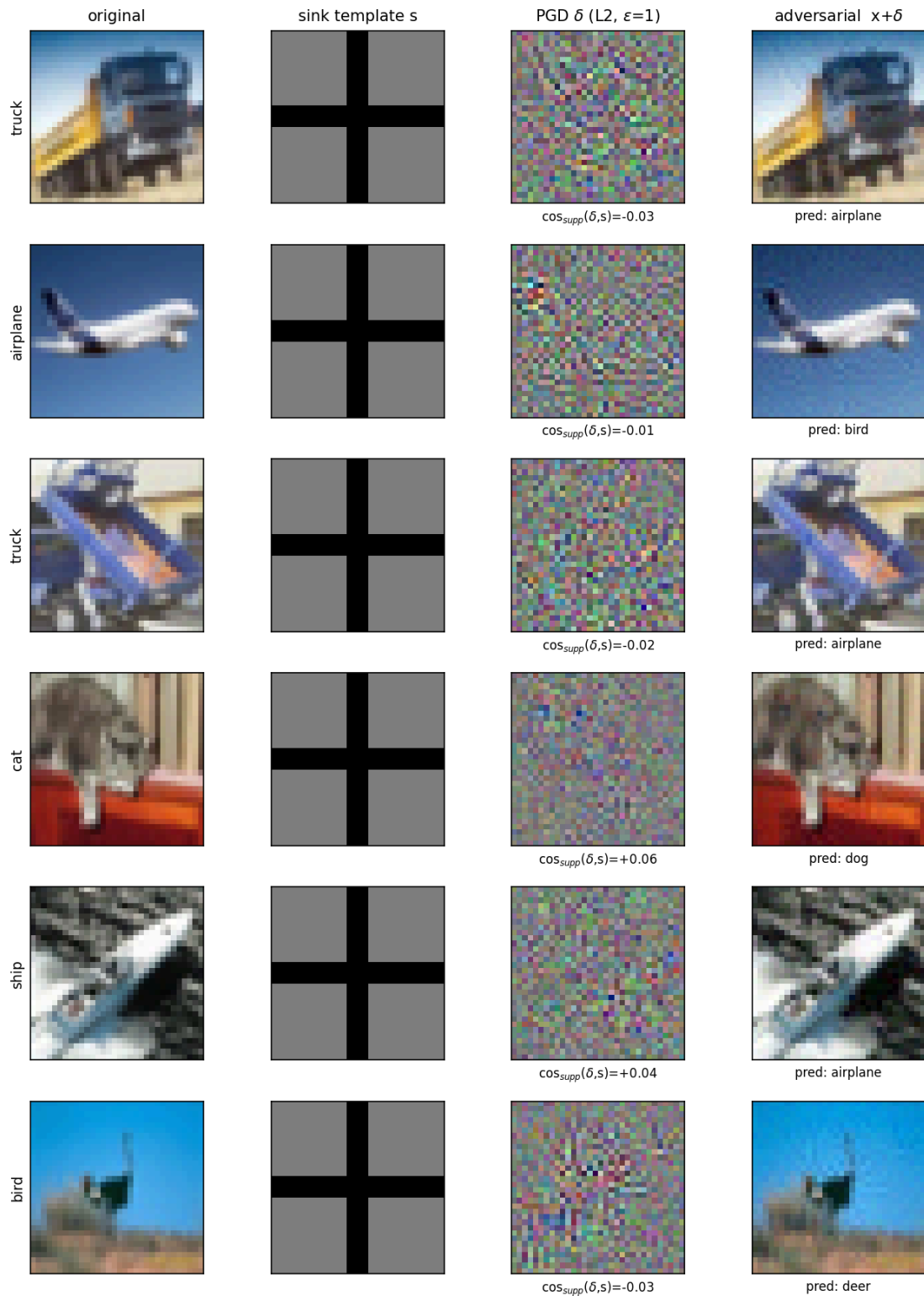
corner_square spadają nawet *poniżej* poziomu losowego, gdyż atak unika narożnika.

Potwierdziliśmy ten wynik bezpośrednio, wizualizując funkcję straty. Gdyby sink był naturalną pułapką dla ataku, przekrój funkcji straty wzdłuż osi sinka powinien wykazywać dolinę - obszar gdzie strata rośnie w kierunku sinka. Taka dolina nie istnieje: funkcja straty jest płaska wzdłuż osi sinka, a trajektoria PGD nigdy nie skręca w jego stronę (Rysunek 10). Zestawienie szablonu sinka z rzeczywistą perturbacją PGD pokazuje, że atak ignoruje wzorec i koncentruje się na pikselach obiektu (Rysunek 11).



Rysunek 10: **Krajobraz strat CIFAR nie ma niecki w stronę sinka.** Dwuwymiarowy przekrój straty klasyfikacji rozpięty przez kierunek sinka i kierunek gradientu wejścia, z nałożoną trajektorią PGD oraz $\cos(\delta_t, s)$ w funkcji kroku ataku. Strata rośnie wzdłuż osi gradientu, lecz jest płaska wzdłuż osi sinka, trajektoria nigdy nie skręca ku sinkowi, a kosinus pozostaje ≈ 0 przez cały czas. Po prostu nie ma ścieżki w dół, którą atak mógłby podążać do sinka.

What the attack draws INSTEAD of the sink: PGD δ is diffuse object-noise, not the cross



Rysunek 11: **Co atak rysuje zamiast sinka.** Kolumny pokazują, dla kilku wejść, czysty obraz, zamierzony szablon sinka oraz rzeczywistą perturbację PGD. Perturbacja jest zorganizowana wokół istotnych pikseli *obiektu* - szum krawędzi i tekstur - i nie przypomina szablonu ($\cos \approx 0$). Atak wydaje budżet tam, gdzie lokalny gradient jest największy, czyli na obiekcie, nigdy na wzorcu projektanta.

FGSM dawał takie same wyniki jak PGD: oba ataki dawały $\text{support_cos} \approx 0$ i mass_frac na poziomie losowym. Różniły się jedynie skutecznością jako ataki - PGD skutecznie oszukiwał model na każdym obrazie, podczas gdy FGSM nie zawsze. Pojawił się jeden pozorny wyjątek: FGSM L_2 dawał mass_frac 0.356, czyli powyżej poziomu losowego 0.234 dla krzyża. Nie wynikało to jednak z rysowania wzorca, lecz z tego, że krzyż leży w centrum obrazu, gdzie sieć naturalnie ma najwyższe gradienty. support_cos pozostawał bliski zera, co potwierdziło, że wzorzec nie był odtwarzany (Tabela 2).

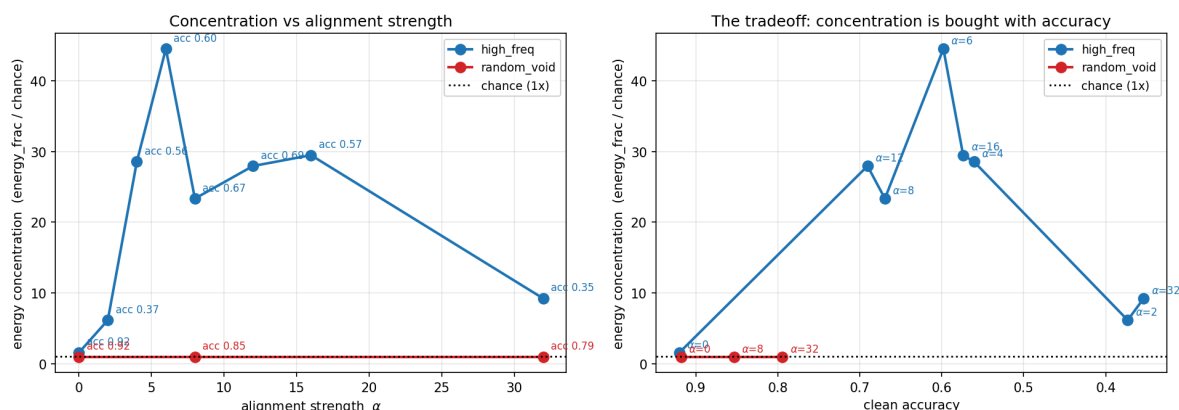
norma	atak	ϵ	dokł. odp.	support_cos	mass_frac
L2	FGSM	0.5	0.236	+0.000	0.356
L2	FGSM	2.0	0.146	-0.002	0.355
L2	PGD	0.5	0.012	+0.002	0.282
L2	PGD	2.0	0.000	+0.004	0.258
L_∞	FGSM	0.031	0.068	+0.006	0.236
L_∞	PGD	0.031	0.000	+0.004	0.238

Tabela 2: **FGSM i PGD są zgodne: brak rysowania, na zbieżnej sieci 0.92.** Dla obu ataków i obu norm $\text{support_cos} \approx 0$, a mass_frac jest na poziomie losowym (0.234 dla krzyża). PGD jest znacznie silniejszy jako atak (dokładność odpornościowa $\rightarrow 0$), lecz nie bardziej sterowalny. Podwyższona masa FGSM L_2 odzwierciedla istotność pikseli centralnych pokrywających się z krzyżem, a nie odtworzenie jego kształtu.

W drugim eksperymencie umieściliśmy sink w kierunku, na który klasyfikator jest naprawdę ślepy - czyli takim, który nie niesie żadnej informacji o klasie obrazu. Wybraliśmy gęsty kierunek wysokoczęstotliwościowy: szachownicę gdzie sąsiednie piksele naprzemiennie przyjmują wartości $+1$ i -1 . Naturalne obrazy niemal nie zawierają takich wzorców, więc klasyfikator ich nie wykorzystuje do rozpoznawania. Jako punkt odniesienia testowaliśmy też kierunek losowy. Dla obu kierunków przeprowadziliśmy przegląd wartości α , sprawdzając jak siła składnika L_{align} wpływa na wyniki.

Koncentracja energii wreszcie przeniosła się na CIFAR, ale tylko dla kierunku wys.,okoczęstotliwościowego i tylko przy odpowiednim α . Model bez składnika L_{align} ($\alpha = 0$) już naturalnie kierował nieco więcej energii ataku na ten kierunek - $1.5 \times$ powyżej poziomu losowego. Zwiększanie α podnosiło ten efekt do szczytu $44 \times$ przy $\alpha = 6$, a przy $\alpha = 8-12$ utrzymywało się na poziomie $23-28 \times$. Dla kierunku losowego α nie dawało żadnego efektu - energy_frac pozostawał na poziomie losowym niezależnie od wartości α . Oznacza to, że koncentracja działa tylko wtedy, gdy sink leży w kierunku na który klasyfikator jest naprawdę ślepy. Sam fakt że kierunek jest „nievisualny” nie wystarczy.

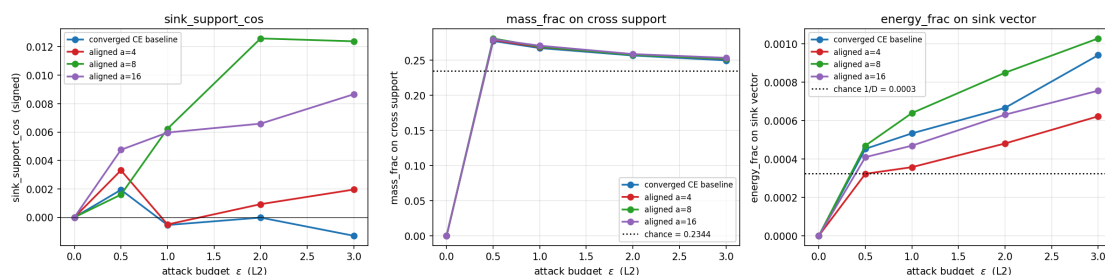
Koszt jest dokładność, a granica jest uderzająco niemonotoniczna. Wraz ze wzrostem α model najpierw się *załamuje* ($\alpha = 2 \rightarrow 0.38$ dokładności, powtarzalnie dla trzech różnych *random seed*), następnie wraca do formy przez $\alpha = 4-12$ (szczyt 0.69), po czym znów się *załamuje* ($\alpha = 32 \rightarrow 0.35$). Nie ma taniego optimum o wysokiej dokładności: każde dopasowanie na tyle silne, by skoncentrować energię, degradowe model z 0.92, a trening stabilizuje się ponownie dopiero przy $\alpha \approx 8-12$.



Rysunek 12: **Koncentracja energii przenosi się na CIFAR-10 tylko dla kierunku ślepego dla etykiety i jest opłacana dokładnością.** Po lewej: koncentracja energii (energy_frac względem poziomu losowego) w funkcji siły dopasowania α . Sink wysokoczęstotliwościowy (niebieski) rośnie od $1.5 \times$ przy $\alpha = 0$ do szczytu $44 \times$ przy $\alpha = 6$, utrzymując $23\text{--}28 \times$ przy $\alpha = 8\text{--}12$; kierunek losowy (czerwony) nigdy nie opuszcza linii losowej. Po prawej: te same punkty względem dokładności, odsłaniające kompromis i jego niemonotoniczność - dokładność spada przy $\alpha = 2$ (0.38), wraca do 0.69 blisko $\alpha = 12$, po czym załamuje się przy $\alpha = 32$ (0.35). Użyteczny obszar pracy to $\alpha \approx 8\text{--}12$: $\sim 23\text{--}28 \times$ poziomu losowego przy dokładności 0.67–0.69.

Mechanizm jest też stabilny względem budżetu ataku na CIFAR (pytanie Q4 z etapu 3): metryki koncentracji utrzymują się przy zmianie ε , więc efekt nie jest artefaktem jednego budżetu (Rysunek 13).

CIFAR-10 sink alignment vs attack budget: aligned fine-tune does not beat the converged baseline at healthy accuracy



Rysunek 13: **Stabilność metryk względem budżetu ataku ε .** support_cos, mass_frac i energy_frac w funkcji budżetu L_2 , dla bazy i sieci dopasowanej. Krzywe są gładkie i monotoniczne, bez efektów progowych: ten (niewielki) sygnał obecny na CIFAR jest spójny w różnych budżetach, a nie pojawia się tylko przy jednym starannie dobranym ε .

5. Synteza i werdykt

Wyniki układają się w trzy główne wnioski.

1. **Rozpoznawalnego wizualnie sinka nie da się narysować na CIFAR.** W pięciu mechanizmach i sześciu wzorcach, na w pełni zbieżnej sieci 0.92 przy szerokości do $2 \times$, żaden wzorec nie został odtworzony (support_cos ≈ 0 , masa na poziomie losowym). Wykluczono zarówno zbyt małą pojemność sieci, jak i zbyt niską wymiarowość jako możliwe przyczyny.
2. **Koncentracja energii ataku na wybrany kierunek jest możliwa, ale tylko pod pewnymi warunkami.** Gdy wybrany kierunek jest taki, że klasyfikator nie używa go do rozróżniania klas, atak PGD skierował $23\text{--}28 \times$ więcej energii na ten kierunek niż wynikałoby z przypadku. Osiągnięto to kosztem dokładności klasyfikacji ($0.92 \rightarrow \approx 0.68$), a zależność od siły składnika L_{align} była niemonotoniczna. Dla kierunku wybranego losowo efekt nie wystąpił.
3. **Eksperymenty na środowisku toy potwierdziły mechanizm.** W prostej sieci na danych 2D, gdzie istniały prawdziwie nieużywane wymiary, koncentracja wyniosła $36 \times\text{--}187 \times$ poziomu losowego.

wego niemal bez strat dokładności i była stabilna przy różnych budżetach ataku. Pokazało to, że efekt koncentracji jest realny - jednak nawet w tym idealnym przypadku znak perturbacji pozostał niekontrolowany: atak mógł skupić energię na danym kierunku, ale nie zagwarantował, że pójdzie w wybraną stronę.

Oba wyniki wyjaśnia jedno spostrzeżenie: *gradient sieci względem wejścia może albo kodować klasę, albo wskazywać ustalony kierunek, ale nie jedno i drugie naraz*. Jeśli kierunek jest naprawdę nieużywany do klasyfikacji, można skupić na nim energię ataku. Jeśli jednak kierunek niesie informację o klasie, gradient musi ją kodować i nie może jednocześnie wskazywać stałego wzorca.

6. Wnioski i kierunki przyszłych prac

Postawiliśmy sobie za cel sprawienie, by model zdradził atakującego, zmuszając ataki gradientowe white-box do narysowania ustalonego symbolu wizualnego. Uczciwy wynik jest taki: silna wersja tego celu jest nieosiągalna z zasadniczych powodów, podczas gdy użyteczna wersja słabsza - zmuszenie ataku do wejścia w znaną, ślepą dla etykiety podprzestrzeń, jest osiągalna i została pokazana zarówno w środowisku toy, jak i na CIFAR-10. Wkładem jest ostra granica między tymi dwoma.

Naturalnie wynika z tego kilka konkretnych kierunków.

1. **Detektor zamiast rysunku.** Osiągalny efekt to dokładnie to, czego potrzebuje detektor: rzutować perturbację wejścia na znaną bazę i sygnalizować anomalną energię. Omija to całkowicie podwójną rolę gradientu i jest najbardziej obiecującym następnym krokiem.
2. **Kształtowanie gradientu poza danymi treningowymi.** Wymóg wysokiej dokładności wiąże model tylko na prawdziwych obrazach, tymczasem PGD prowadzi atak daleko poza nie. Stąd pomysł: utrzymać zachowanie modelu na prawdziwych obrazach składnikiem KL względem zamrożonego, dobrze wytrenowanego klasyfikatora (co chroni dokładność), a składnik L_{align} stosować wyłącznie w punktach, do których prowadzi atak. Oba cele przestają wtedy rywalizować o ten sam gradient. Sprawdziliśmy to w środowisku toy. Składnik KL rzeczywiście utrzymał wysoką dokładność na czystych przykładach (0.974 - więcej niż przy zwykłym dopasowaniu, które przy silniejszym wymuszaniu się załamuje), ale rozdzielenie punktów nie przełożyło się na sterowanie atakiem: $\cos(\delta, s)$ spadł nieznacznie poniżej zera (-0.084), podczas gdy najlepsze zwykłe dopasowanie dawało $+0.17$, a wariant z różniczkowalnym PGD w ogóle się rozbiegł (Rysunek 7). Rozdzielenie celów w przestrzeni chroni więc dokładność, ale nie usuwa samego napięcia. Pozostaje otwarte, czy staranniej dobrana wersja w skali CIFAR mogłaby pomóc.
3. **Trening wyznaczonego UAP i wąskie gardła architektury.** Trenować sink jako ustalony, kontrolowany co do tempa, uniwersalny kierunek zwiększający stratę [5] (łagodniejszy niż wymuszona etykieta), albo zbudować niskorangowe wąskie gardło wejścia, którego atakowalna podprzestrzeń zawiera sink z konstrukcji.
4. **Ujęcie przez cechy odporne.** Fakt, że tylko kierunek ślepy dla etykiety koncentruje energię, łączy się wprost ze spojrzeniem na przykłady adversarialne przez cechy odporne i nieodporne [6]; scharakteryzowanie, które podprzestrzenie są „darmowe” dla danego zbioru, pozwoliłoby z góry przewidzieć osiągalną koncentrację.

Bibliografia

- [1] Ian J. Goodfellow, Jonathon Shlens, i Christian Szegedy, „Explaining and Harnessing Adversarial Examples”, w *International Conference on Learning Representations (ICLR)*, 2015.
- [2] Jonas Rauber, Wieland Brendel, i Matthias Bethge, „Foolbox: A Python Toolbox to Benchmark the Robustness of Machine Learning Models”, 2017.
- [3] Aleksander Mądry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, i Adrian Vladu, „Towards Deep Learning Models Resistant to Adversarial Attacks”, w *International Conference on Learning Representations (ICLR)*, 2018.
- [4] Tianyu Gu, Brendan Dolan-Gavitt, i Siddharth Garg, „BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain”, 2017.
- [5] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, Omar Fawzi, i Pascal Frossard, „Universal Adversarial Perturbations”, w *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [6] Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, i Aleksander Mądry, „Adversarial Examples Are Not Bugs, They Are Features”, w *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] Christian Szegedy *et al.*, „Intriguing Properties of Neural Networks”, w *International Conference on Learning Representations (ICLR)*, 2014.
- [8] Alex Krizhevsky, „Learning Multiple Layers of Features from Tiny Images”, University of Toronto, 2009.